

A4 GraphColoring

```
class GraphColoring {  
    private int V; // Number of vertices  
    private int[][] graph; // Adjacency matrix  
    private int[] colors; // Colors assigned to vertices  
    private String[] colorNames = {"", "Red", "Green", "Blue"}; // Mapping for color codes  
  
    public GraphColoring(int[][] adjacencyMatrix, int numColors) {  
        V = adjacencyMatrix.length;  
        graph = adjacencyMatrix;  
        colors = new int[V]; // Default initialized to 0 (no color)  
    }  
  
    // Function to check if it's safe to color a vertex  
    private boolean isSafe(int v, int c) {  
        for (int i = 0; i < V; i++) {  
            if (graph[v][i] == 1 && colors[i] == c) {  
                return false;  
            }  
        }  
        return true;  
    }  
  
    // Backtracking function to color the graph  
    private boolean solveGraphColoring(int v, int m) {  
        if (v == V) {  
            printSolution();  
            return true;  
        }  
    }  
}
```

```

    for (int c = 1; c <= m; c++) {
        if (isSafe(v, c)) {
            colors[v] = c;
            if (solveGraphColoring(v + 1, m)) {
                return true;
            }
            colors[v] = 0; // Backtrack
        }
    }
    return false;
}

// Function to print the color assignment with names
private void printSolution() {
    System.out.println("Vertex Colors:");
    for (int i = 0; i < V; i++) {
        System.out.println("Vertex " + i + " -> " + colorNames[colors[i]]);
    }
}

// Function to start the coloring process
public void solve(int m) {
    if (!solveGraphColoring(0, m)) {
        System.out.println("No solution exists.");
    }
}

// Main function to run the program
public static void main(String[] args) {

```

```
int[][] graph = {  
    {0, 1, 1, 1},  
    {1, 0, 1, 0},  
    {1, 1, 0, 1},  
    {1, 0, 1, 0}  
};  
  
int numColors = 3; // Red, Green, Blue  
  
GraphColoring gc = new GraphColoring(graph, numColors);  
gc.solve(numColors);  
}  
}
```

Output

Vertex Colors:

Vertex 0 -> Red

Vertex 1 -> Green

Vertex 2 -> Blue

Vertex 3 -> Green